



Apresentação do GIT

Características, Exemplo de uso e Funções

Outros exemplos

1. Criar um repositório local e clonar da nuvem

Este são os primeiros processos que serão realizados em um projeto.

Criar repositório local

git init: Inicializa um repositório Git na pasta atual. A partir daqui, o Git começa a rastrear alterações.

```
aluno@S309-001 MINGW64 ~  
$ git init repositorio_teste  
Initialized empty Git repository in C:/Users/aluno/repositorio_teste/.git/
```

Outros exemplos

Clonar um repositório remoto

git clone <https://github.com/usuario/projeto.git>: Baixa um repositório da nuvem (por exemplo, do GitHub) para sua máquina, incluindo todo o histórico.

```
aluno@S309-001 MINGW64 ~  
$ git clone https://github.com/lhmarques2002-jpg/prjens_ifg_gyn_26.git  
Cloning into 'prjens_ifg_gyn_26'...  
remote: Enumerating objects: 342, done.  
remote: Counting objects: 100% (93/93), done.  
remote: Compressing objects: 100% (72/72), done.  
remote: Total 342 (delta 56), reused 21 (delta 21), pack-reused 249 (from 2)  
Receiving objects: 100% (342/342), 10.46 MiB | 22.79 MiB/s, done.  
Resolving deltas: 100% (128/128), done.
```

Outros exemplos

2. Colocar um item na área de preparo e confirmar

Após o projeto alcançar um desenvolvimento apropriado, se é recomendado realizar um commit para que o presente estado do arquivo seja salvo no repositório e antes que isso seja feito é necessário movê-lo para a área de preparo,

git add arquivo.txt: envia o arquivo para a *staging area* (área de preparo)

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26/mat_did/Material GIT (main)
$ git add .

aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26/mat_did/Material GIT (main)
$ |
```

Outros exemplos

git commit -m "Adiciona arquivo": salva definitivamente a alteração no histórico do projeto

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:
#   deleted:   conteudo.txt
#
# Untracked files:
#   ../../teste.txt
#
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:
#   deleted:   conteudo.txt
#
# Untracked files:
#   ../../teste.txt
#
```

Outros exemplos

3. Pegar alterações da branch principal e mesclar com outra branch

Quando se faz trabalhos em equipe é previsto que seja necessário pegar alterações feitas por outros membros do grupo e mesclar as mesmas com as que estão sendo feitas localmente.

Atualizar a principal

git checkout main: muda para a branch principal

git pull: baixa e integra alterações do repositório remoto

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (main)
$ git checkout main
D      mat_did/Material GIT/conteudo.txt
Already on 'main'
Your branch is up to date with 'origin/main'.
```

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (main)
$ git pull
Already up to date.
```

Outros exemplos

Ir para sua branch e mesclar

git checkout minha-branch: Volta para sua branch

git merge main: traz as atualizações da principal para sua branch

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (main)
$ git checkout branch1
D      mat_did/Material GIT/conteudo.txt
Switched to branch 'branch1'
```

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch1)
$ git merge main
Already up to date.
```

Outros exemplos

4. Resetar arquivo sem perder alterações

Quando se vê que a ideia testada em um ramo *não dará frutos imediatos*, é às vezes *necessário momentaneamente guardar aquele ramo e voltar ao trabalho principal*.

Opção segura para guardar alterações

git stash: Salva temporariamente todas as suas alterações (como se colocasse numa gaveta).

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch1)
$ git stash
Saved working directory and index state WIP on branch1: 7ad0147 Add files via up
load
```

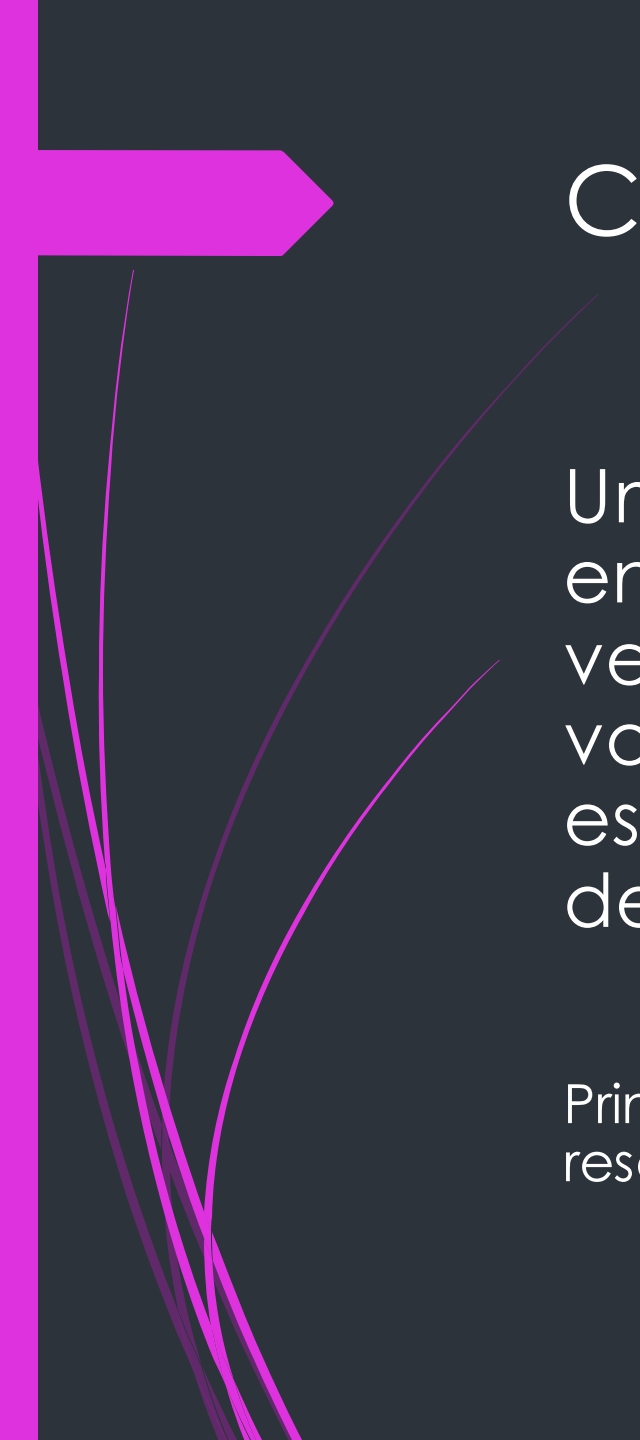
Outros exemplos

Restaurar o arquivo ao estado original

git checkout main -- arquivo.txt: Pega a versão do arquivo da branch principal e substitui a atual.

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch1)
$ git checkout main -- mat_did

aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch1)
$
```



Considere um problema

Um erro crítico é descoberto no código principal enquanto se está trabalhando em uma nova versão de um software, tornando necessário voltar ao arquivo principal sem perder o que se está trabalhando e fazer as alterações sem risco de danificar ainda mais o código, isso é possível?

Primeiro, vamos entender a ferramenta que iremos utilizar para resolver o problema.

Exemplo prático

Voltando ao problema do início da apresentação:

Aqui estão os passos que serão seguidos para resolver o problema:

1. Salvar temporariamente o que você está fazendo

git stash: Guarda temporariamente suas alterações não confirmadas e limpa o diretório de trabalho, permitindo mudar de contexto sem perder nada.

2. Voltar para a branch principal

git checkout main: Muda para a branch principal do projeto, onde está o código “oficial”.

Exemplo prático

3. Criar uma nova branch para corrigir o erro

git checkout -b hotfix-erro-critico: Cria e já muda para uma nova branch. Isso isola a correção sem afetar o código principal diretamente.

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (main)
$ git checkout -b branch2
Switched to a new branch 'branch2'
```

4. Corrigir o erro e salvar

git add: prepara as alterações para commit

git commit -m "Corrige erro crítico": salva a correção no histórico

Exemplo prático

5. Aplicar a correção na branch principal

git checkout main: Volta para a branch principal

git merge hotfix-erro-critico: Junta (*merge*) a correção feita na branch de hotfix

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch2)
$ git merge branch2
Already up to date.
```

Exemplo prático

6.(Opcional) Apagar a branch de correção

git branch -d hotfix-erro-critico: Remove a branch temporária após o uso.

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (branch2)
$ git branch -d branch1
Deleted branch branch1 (was 7ad0147).
```

7. Voltar ao que você estava fazendo antes

git checkout sua-branch: Retorna para sua branch de desenvolvimento

git stash pop: recupera o que você estava fazendo antes

```
aluno@S309-001 MINGW64 ~/repositorio_teste/prjens_ifg_gyn_26 (main)
$ git stash pop
No stash entries found.
```



O que foi feito

Você:

- 1) Guardou o trabalho atual (*stash*)
- 2) Foi para a ramificação principal
- 3) Criou uma *branch* de correção
- 4) Fez a correção e fez um *commit*
- 5) Integrou no arquivo principal (*merge*)
- 6) Excluiu a ramificação de correção do erro
- 7) Voltou ao seu trabalho original

Referências

- FLEURY, Cláudio A.; "Git – Olhando sob o capô"; 2026; disponível em: https://github.com/kaw19/prjens_ifg_gyn_26/tree/main/mat_did; acesso em 26/04/2026.
- FLEURY, Cláudio A.; "Git "(formato pdf); 2026; disponível em: https://github.com/kaw19/prjens_ifg_gyn_26/tree/main/mat_did; acesso em 26/04/2026.
- HOSTINGER (site); "Comandos Git"; disponível em: <https://www.hostinger.com/br/tutoriais/comandos>; acesso em 27/04/2026.
- Apoio de ferramentas de IA: ChatGPT e Gemini.